Alex Castillo González

Applied AI Engineer · Python / LLM

Santo Domingo, Dominican Republic — UTC−4 · Independent contractor (Spanish passport) · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alexcastillogonzalez · portfolio-alexgonzalez33.vercel.app

SUMMARY

Applied AI engineer working in Python on retrieval and agent systems, and on the layer that decides whether they survive real users: evaluation, failure handling, and knowing which numbers actually moved. My main retrieval project answers over the text of the **GDPR** and is built around a constraint that regulated domains impose and generic RAG ignores — every statement has to name the provision it came from, and the system has to decline when the source does not cover the question. Both projects ship with the harness that measures them — retrieval, citation accuracy and abstention for the RAG system, tool trajectories and answer grounding for the MCP agent — and in each case the harness found defects the tests did not. That measurement layer is now a published library, `ragcite`, which asserts that it reproduces the numbers of the project it was extracted from. Fullstack background across Python, TypeScript and Ruby, with production experience shipping and operating what I build. I also fix the frameworks this work runs on: five merged fixes in **Haystack**, deepset's framework for production RAG and agent pipelines — four concurrency defects on its async path, one serialization defect that changed how a component behaved after a reload; a merged fix to `pydantic-ai`'s eval framework; five open fixes to the retrieval evaluation and MMR code in `llama-index-core`; and two merged in `pyfenn/fenn`.

SKILLS

AI / LLM — Python · LLM APIs (Groq, OpenAI-compatible) · LangChain · LangGraph · ReAct agents and tool use · **Model Context Protocol (MCP)**: building servers and clients · **agent trajectory evaluation** (tool selection, ordering, argument accuracy, answer grounding) · RAG: chunking strategy, retrieval tuning, **structure-aware citation at the provision level** · **LLM evaluation**: known-answer datasets, retrieval metrics (hit@1, recall@k, MRR), **abstention and unsupported-claim rate**, LLM-as-judge for faithfulness / relevancy / correctness / context precision · cross-lingual retrieval · resilience to unreliable model output (retry with rate-limit backoff, graceful degradation)

Regulatory / legal text — parsing legislative sources into citable provisions (EUR-Lex / Official Journal markup) · citation integrity as a design constraint · evaluating refusal on out-of-corpus questions · GDPR structure (data subject rights, controller and processor obligations, breach notification, administrative fines)

Data & retrieval — embeddings (`sentence-transformers`, BGE, MiniLM) · FAISS · PostgreSQL + `pgvector` · PDF ingestion pipelines · structured corpus construction

Backend — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API)

Web — TypeScript · Next.js · React · Tailwind · JavaScript · HTML/CSS/SCSS

Infra & tooling — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch, Budgets) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Docker Compose · Vercel · Streamlit Community Cloud · Git (GitHub, GitLab) · AI-assisted development (Claude Code, Cursor)

EXPERIENCE

Churrería Calderón — Family business · Toronto, Canada

Oct 2025 – Jul 2026 (business closed July 2026)

- **Developer:** built and deployed the business website, plus an embeddable AI chat widget that answered customer questions on menu, hours, location and FAQs, grounded only in the business's own information so it would not invent details. Built the widget to be reusable across clients from a single configuration file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.

- Shipped both during the setup period before the December 2025 opening, so the site and the assistant were live on day one rather than added later.
- Worked in day-to-day operations of the business throughout, which is where the judgement about which problems are worth automating — and which are not — came from.

Family churrería business — Owner-operator · Catalonia, Spain

2023 – 2026 (and in the same business before that)

- Ran the business single-handed: production, service, customers, purchasing, cash and compliance. No staff to delegate to and no manager to escalate to — if it did not work, it was mine to fix that morning.
- Took it from a mobile trailer to fixed premises, which meant rebuilding the operation around a different site, different hours and a different customer base.
- Also worked my father's stand on the fairground circuit.
- Years of reading what customers actually ask for, in person, hundreds of times a day. That is the same instinct a support-facing product feature needs, and it is why the AI widget I later built was scoped to the four questions people really ask.

Le Wagon — Part-time Programming Teacher · Remote (France)

Oct 2022

- Taught on the new part-time flex cohort of the fullstack bootcamp: Ruby, object-oriented programming, SQL and PostgreSQL, HTML/CSS/JavaScript, and building on Ruby on Rails.
- Supported students through exercises and debugging during live sessions.

TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

Aug 2022 – Nov 2022 · On-site

- Shipped features into a long-running internal application maintained by a team of four engineers and a systems engineer.
- **Backend:** wrote Ruby routines that pulled data from JSON files and the **Jira API**, transformed it and persisted it to PostgreSQL; implemented multi-stage validation processes that gated document generation and downloads.
- **Frontend:** built form-driven pages and PostgreSQL-backed views with role-dependent data (users and managers saw different data), and surfaced the state of backend subprocesses so users could follow a document download to completion.
- Worked across a codebase distributed over both GitHub and GitLab.

SELECTED PROJECTS

Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules: tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result. The grounding check needs no judge model and is therefore deterministic and free — it holds even when a fabricated number happens to be correct.
- The harness caught the agent answering "I don't have that information" with zero tool calls, and caught it **intermittently**, which single-run testing misses; scenarios can be repeated to measure flaky behaviour. Scored 11/12, mean 0.98.
- Chose not to use `langchain-mcp-adapters`: it pins `mcp<2` and would have forced a working server back to an older SDK. Wrote a 60-line bridge instead, passing the server's JSON Schema straight through so no tool signature is duplicated.

- **Deployed it to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production with no change above the backend.
- CI/CD on every push to main via GitHub Actions, authenticating with **OIDC** rather than a stored access key, and scoped to one branch so a fork's pull request cannot assume the deploy role. The role deliberately cannot apply infrastructure: it can ship an image and repoint the function, nothing more.
- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, GitHub Actions, pytest (161 tests).

Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system declines when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting, and a reader sent to "page 14" can confirm nothing. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex (not a mirror — in a system whose claim is checkable citations, the text has to come from the authority the citation names) into **414 provisions** carrying article, paragraph, title and chapter as metadata. The chunk size was then chosen so that **97% of provisions survive as exactly one chunk**, because a chunk straddling Art. 33(1) and 33(2) gets attributed to one of them and cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent, confident and indistinguishable from a correct answer, and a citation the model reasoned its way to rather than read makes it more convincing. So the harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions by country, the text of the standard contractual clauses, Schrems II, a CCPA penalty — alongside a deterministic check for citations that appear in the answer but never in the retrieved passages. Every question scored so far declined correctly and specifically, naming the provision that creates the mechanism and stating that the detail asked for is not in the text; none produced an unsupported answer or a citation that was never retrieved.
- Changed the ground truth from matching text to **matching the cited provision**, which is stricter (retrieving the right words is not retrieving the right authority) and removes the chunk-boundary false misses the substring approach suffered.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading measurably hurt retrieval under the old embedding model and measurably helped under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap itself was worth 8/20 → 13/20 at rank 1 for 42 MB of RAM, against a hard 1 GB ceiling measured with the index loaded, not just the model.
- Three parse bugs caught by sanity checks that now block the build: the closing article silently absorbing the signatures and all 21 footnotes, nested sub-points truncated by a non-greedy regex that could not handle nested markup, and the 26 definitions of Art. 4 collapsing into one uncitable record.
- Fixed cross-lingual retrieval (Spanish 0/4 at rank 1 against an English index) by translating the retrieval query rather than paying ~350 MB for a multilingual embedding model that did not fit the 1 GB budget — a measured trade-off.
- CI on every push runs the suite headlessly, including a boot test of the app itself — the host redeploys straight from `main`, so the suite is the only gate before the public demo. One test forces the embedding loader to raise and asserts the first render still succeeds, proving nothing heavy sits on the render path.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests).

ragcite — the evaluation layer of the two projects above, as a library

Code

Retrieval metrics, citation grounding, abstention scoring and tool-trajectory scoring, extracted from the two harnesses above and generalized. No LangChain, vector store or LLM client is imported: you bring the retriever and the judge, `ragcite` scores what they return.

- **It reproduces the numbers of the project it came from, and asserts it.** The dogfood example scores one FAISS index twice — once with `ragcite`, once with the original project's own independent metrics code — and exits non-zero if they disagree. They match exactly (hit@1 13/25, recall@k 17/25, MRR 0.59), which is what makes "extracted from a production harness" a check rather than a claim.
- Retrieval and grounding need no model at all: `hit@1` / `recall@k` / `MRR` are id matching and grounding is a set membership test, so both are deterministic and free to run on every commit. A fabricated citation is caught however fluently it reads.
- `--min-hit-at-1`, `--min-recall`, `--min-mrr` turn a run into a CI gate that exits 2 on a regression. Without a threshold flag the command always exits 0 — a report, not a gate, and it stays that way by default.
- `check_judge_independence` refuses to run when the judge and the system under test are the same model. That defect is why it exists: my own harness spent weeks marking a model as its own examiner.
- Zero runtime dependencies, MIT, 155 tests, CI on Python 3.9–3.12.

AI Website Chat Widget — embeddable business assistant

Live demo · Code

Drop-in chat widget for small-business sites, grounded strictly in the business's own content. Reusable for any client from a single config file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.

Semantic Recommender — embedding-based recommendations

Code

Recommendation engine built on vector embeddings with a feedback loop that refines results over time, as a reusable backend for platforms that have outgrown rule-based filters. Python, embeddings, pgvector, PostgreSQL.

OPEN SOURCE

Merged

- `deepset-ai/haystack` #12364 — `LinkContentFetcher` rotated its `User-Agent` on a cursor held by the component, but `run()` fetches the URLs concurrently: a retry triggered by one URL advanced the user agent for all the others, and each completed fetch reset the cursor underneath the requests still in flight, so most retries went out un-rotated. Each fetch now walks the list on its own. Closed the upstream issue.
- `deepset-ai/haystack` #12359 — `LLMDocumentContentExtractor.run_async` converted every document to an image inline: reading files from disk, rendering PDF pages and base64-encoding them on the event loop before the first LLM call was scheduled.
- `deepset-ai/haystack` #12358 — `EmbeddingBasedDocumentSplitter.run_async` was only async for its first pass: the recursive re-split of over-long chunks called the blocking embedder, running the most expensive part of the work on the event loop. Shipped in the 3.1 milestone.
- `deepset-ai/haystack-core-integrations` #3790 — `OAuthRefreshTokenSource` kept one `asyncio.Lock` for the life of the source. That lock binds to the loop that first awaits it under contention and raises on any other, so a source reused across event loops — one `asyncio.run` per request is a common deployment — failed on the second loop's first contended refresh. I reported it as issue #3789 and closed it with this PR.
- `deepset-ai/haystack-core-integrations` #3808 — three components took an `__init__` parameter, used it at run time and left it out of `to_dict`, so the value silently reverted to its default once a pipeline was saved and reloaded, with nothing in the serialized dict to show it had ever been set: a zero-shot router losing the flag that decides how its label scores are normalised — and it picks its output branch from those scores, so the same text can route elsewhere after a round trip; a TEI ranker that stops asking the endpoint for raw scores; and a Ragas evaluator falling from 16 concurrent LLM judgements back to 4. Found by auditing `to_dict` against `__init__` across the integrations, not from an issue.

- `pydantic/pydantic-ai` #7936 — `pydantic-evals` reads `expected_output=None` as "no expectation", so a `Case` written to assert that a task returns `None` is skipped instead: `EqualsExpected` records no assertion and the case averages 1.0 whether the task returns `None` or the wrong answer outright. The sentinel and the legitimate value are the same object. The maintainers hold the skip as intended, so the fix is documentation — the trap is now stated where the behaviour is defined, and `Equals(value=None)` named as the evaluator that does assert it. Reported as #7934 with a runnable reproduction; closed by this PR.
- `pyfenn/fenn` #277 — added `.docx` support to the RAG document loader, so the framework ingests Word documents alongside PDFs and text.
- `pyfenn/fenn` #286 — corrected the RAG optional-dependency install instructions, which referenced a package name that does not exist.
- `rubocop/rubocop-rspec` #2209 — `RSpec/LeadingSubject` crashed on Ruby 3.4's implicit `it` block parameter: the cop walked `:block` AST ancestors only, so an example group written as an `itblock` or `numblock` was never found and the lookup returned `nil`. Widened to `:any_block`, with a regression spec pinned to Ruby 3.4.
- `Rails-Designer/courrier` — five merged in a Ruby mailer gem, all shipped in its 1.1.0 release: MailerSend, Mailtrap and SMTP.com provider integrations, which closed the gem's standing request for more providers; a `NameError` that broke Mailgun and Mailjet on Ruby 3.4 — `Base64` left the default gems and those two were the only providers calling it without requiring it, so the gem installed fine and raised on send; and `cc:`/`bcc:` delivery, which the gem accepted on every email while six providers silently dropped it. I reported that one as #58; the maintainer asked for the PR. Each provider now takes the copies in the shape its own API wants, which for SparkPost is not a field at all — every copy is a recipient there, and what separates a cc from a bcc is whether the address repeats in the CC header. Reading the lists through one helper also fixes Mailjet, SendGrid and SparkPost sending several `to:` addresses as a single malformed one — filed as #59.

Each Haystack fix ships a regression test I verified fails with the fix reverted, rather than passing either way. I wrote the four concurrency ones up together, because they are one class of defect and three were invisible to the test suite for the same reason: Four concurrency bugs on Haystack's async path.

Open

- `deepset-ai/haystack-core-integrations` #3873 and `deepset-ai/haystack` #12518 — the same defect as #3808, found again by scripting the audit: a small AST pass comparing every component's `__init__` parameters against the keys that reach `to_dict`. Four more settings were being dropped, two in the `transformers` and `amazon_bedrock` integrations and two in Haystack itself, each one a value a reloaded pipeline goes on using at its default with nothing to show it was ever set: the botocore config behind an S3 downloader's timeouts and retries, the threshold deciding which overlapping answers an extractive reader discards. A sibling component already serialized the parameter in each case, and the existing tests showed the omission was an oversight — one was parametrized over a value that could not change its own assertion. The audit's first version also flagged `google_vertex`; a maintainer pointed out on my issue #3874 that the integration is archived, so I dropped that commit and the PR now covers the two active ones. A later run of the same audit caught #3923: `NvidiaGenerator` drops its request `timeout` from `to_dict` while its four sibling Nvidia components serialize it, so a reloaded pipeline silently falls back to the 60s default.
- `run-llama/llama_index` — five fixes in `llama-index-core`, found by reading the retrieval and evaluation code rather than from an issue. #22683: the retrieval metrics scored outside their own range when a ranking repeated a node id, which is what fusion retrievers produce — hit rate and average precision returned 2.0, NDCG 1.63, so a mean over an eval set stopped being comparable between runs. #22684: MMR discounted each candidate only against the result selected immediately before it, so a near-duplicate re-entered the ranking as soon as an unrelated result was picked in between. #22685: the multi-modal evaluator scored image nodes as text results, because `ImageNode` subclasses `TextNode` and the two type checks were independent. #22968: `default_parser`, the default parser for the judge output behind `CorrectnessEvaluator`, split a two-line response and raised `ValueError` — aborting the eval run — when the judge returned a score with no reasoning line. #22969: `CohereRerankRelevancyMetric` guarded a missing API-key variable with `except IndexError`, but a missing environment variable raises `KeyError`, so the intended "pass in an API key" error never replaced the bare traceback. Each ships with a test that fails without the fix.
- `rubocop/rubocop-rspec` #2214 — fixed `RSpec/LeadingSubject` autocorrecting a subject to a position above another subject.

- `rubocop/rubocop-performance` #529 — fixed `Performance/ConstantRegexp` emitting invalid code when autocorrecting a regexp used as a pattern in `case`/`in` pattern matching.
- `rubyforgood/human-essentials` #5656 — a Rails inventory app for nonprofit essentials banks. Its participant drop-downs displayed one column and were ordered by another, so the list was sorted on a value the user could not see, and the tie-break fell through to the cluster's collation, which does not match between CI and production. Now ordered on the name actually rendered, with runs of digits compared by value so "Store 9" precedes "Store 10". Brakeman flagged my first version as SQL injection for interpolating into `Arel.sql`, so the expression is a literal constant with nothing interpolated. The maintainer also asked for a survey of every other drop-down in the app, which I traced from each rendered `<select>` back to the query that builds it. **Reported**

Defects found by reading the code, filed with a standalone reproduction rather than a bug report someone else has to reproduce first.

- `pydantic/pydantic-ai` #7927 — `pydantic-evals` renders a section of the judge's prompt by iterating anything that is a `Sequence` and is not a `str`. `bytes`, `bytearray` and `memoryview` are all `Sequence`s, so an eval task returning binary content had it rendered as one decimal byte value per line: the judge graded `84 104 101 …` against the rubric and returned a plausible score for content it never saw. No exception, no warning — the failure an eval framework must not have. Reproduced through the public API with no provider key, using a stub model to capture the prompt. Accepted for implementation.
- `pydantic/pydantic-ai` #7928 — the same report's averages give no denominator: an evaluator that scored 1 case of 4 renders identically to one that scored all 4, and `report.averages()` — which the documentation recommends for comparing implementations and validating changes before deployment — exposes only the number. A judge exhausting its quota mid-run therefore reports a higher score than the run earned, and a threshold check passes. Routed to maintainer discussion.
- `deepset-ai/haystack` #12519 — `main` was failing on every pull request because an `openai` release added three fields to its usage models and two tests assert an exact usage dict. Bisected to the version, filed with the reproduction; a maintainer merged the fix the same day.
- `deepset-ai/haystack-core-integrations` #3789 — the `asyncio.Lock` cached across event loops described above. Triaged `P3` by the maintainers; closed by #3790.
- `pyfenn/fenn` #285 — RAG optional-dependency errors pointing at a package and extras that do not exist; closed by #286 above.

EDUCATION

Le Wagon — Fullstack Web Development bootcamp · 2022 Ruby, Ruby on Rails, JavaScript, SQL/PostgreSQL, HTML/CSS.

Self-directed, 2022 – 2025 — LaunchSchool coursework, coding challenges and independent study alongside running the business, before moving back into engineering full time.